```
/* ============================================================================
   HEALTHCONNECT RWANDA - HEALTHCARE ANALYTICS DATABASE
   AnalystLab Africa Experience Lab | Week 6 | Data Analytics track
   Project : Healthcare Operations and Performance Analytics Dashboard
   File    : HealthConnect_Week6_SQL_Script.sql
   Author  : Emmanuel
   Engine  : PostgreSQL 14 or later (tested on PostgreSQL 16)
   ----------------------------------------------------------------------------
   PURPOSE
     1. Build the HealthConnect relational database (schema, keys and constraints
     2. Load and validate the source data.
     3. Answer the 40 business questions set in the assignment.
     4. Reconcile the key figures with the Power BI dashboard.

   HOW TO RUN
     a) Run  python prepare_csv_for_postgres.py  once. It writes the Excel sheets
        gathers every source file in one folder called  csv_for_postgres.
     b) Open a terminal INSIDE that folder, then run:
            psql -U postgres -d healthconnect -f HealthConnect_Week6_SQL_Script.s
        (create the database first with:  CREATE DATABASE healthconnect;)
        The \copy commands in Part 2 read files from the current folder.
     c) pgAdmin users: run Part 1, import each CSV into its stg_ table with the I
        tool (table names are listed in Part 2), then run Part 3 onwards.
     The script can be re-run safely; it rebuilds the hc and stg schemas each tim

   SCOPE AND DATA NOTES (read before interpreting results)
     * The Appointments table holds 5,000 appointments. Billing, payments, claims
       laboratory, prescription, admission and referral files contain many more r
       can be linked to those appointments. Only records that link to one of the
       appointments are loaded into the hc schema; everything else stays in stg a
       counted in the reconciliation query (3.2). This is the same rule the Power
     * No patient master table, facility master table or patient-to-village link
       Queries that need them (gender, age, blood group, phone, province) are wri
       an optional hc.patients table and return no rows until it is loaded. Where
       alternative can be answered from the supplied data, it is provided and lab
     * All amounts are in Rwandan francs (RWF, per the Countries reference table)
     * Cash received is taken from payments.amount_paid. Payment and billing stat
       not reliable indicators of what was actually paid (see check 3.3, item 9).
   ============================================================================

/* ============================================================================
   PART 1 | SCHEMA
   ============================================================================
```

```
DROP SCHEMA IF EXISTS hc  CASCADE;
DROP SCHEMA IF EXISTS stg CASCADE;
CREATE SCHEMA hc;     -- clean, validated tables used for analysis
CREATE SCHEMA stg;    -- raw landing tables loaded straight from the files

SET search_path = hc, public;

-- 1.1 Geography (reference data) ---------------------------------------------
CREATE TABLE hc.countries (
    country_id    VARCHAR(10) PRIMARY KEY,
    country_name  VARCHAR(60) NOT NULL,
    iso2          CHAR(2),
    iso3          CHAR(3),
    currency      CHAR(3),
    region        VARCHAR(40)
);

CREATE TABLE hc.provinces (
    province_id    VARCHAR(10) PRIMARY KEY,
    province_name  VARCHAR(60) NOT NULL,
    country_id     VARCHAR(10) NOT NULL REFERENCES hc.countries
);

CREATE TABLE hc.districts (
    district_id    VARCHAR(10) PRIMARY KEY,
    district_name  VARCHAR(60) NOT NULL,
    province_id    VARCHAR(10) NOT NULL REFERENCES hc.provinces
);

CREATE TABLE hc.sectors (
    sector_id    VARCHAR(10) PRIMARY KEY,
    sector_name  VARCHAR(60) NOT NULL,
    district_id  VARCHAR(10) NOT NULL REFERENCES hc.districts
);

CREATE TABLE hc.villages (
    village_id    VARCHAR(10) PRIMARY KEY,
    village_name  VARCHAR(60) NOT NULL,
    sector_id     VARCHAR(10) NOT NULL REFERENCES hc.sectors
);

-- 1.2 Facilities, departments and capacity -----------------------------------
-- No facility master file was supplied, so facilities are derived from the depar
CREATE TABLE hc.facilities (
    facility_id  VARCHAR(10) PRIMARY KEY
);
```

```
CREATE TABLE hc.departments (
    department_id    VARCHAR(10) PRIMARY KEY,
    department_name  VARCHAR(60) NOT NULL,
    facility_id      VARCHAR(10) NOT NULL REFERENCES hc.facilities
);

CREATE TABLE hc.specialties (
    specialty_id   VARCHAR(10) PRIMARY KEY,
    specialty      VARCHAR(60) NOT NULL,
    department_id  VARCHAR(10) NOT NULL REFERENCES hc.departments
);

CREATE TABLE hc.services (
    service_id     VARCHAR(10) PRIMARY KEY,
    service_name   VARCHAR(80) NOT NULL,
    department_id  VARCHAR(10) NOT NULL REFERENCES hc.departments
);

CREATE TABLE hc.equipment (
    equipment_id    VARCHAR(10) PRIMARY KEY,
    equipment_name  VARCHAR(80) NOT NULL,
    department_id   VARCHAR(10) NOT NULL REFERENCES hc.departments,
    facility_id     VARCHAR(10) NOT NULL REFERENCES hc.facilities
);

CREATE TABLE hc.rooms (
    room_id        VARCHAR(10) PRIMARY KEY,
    department_id  VARCHAR(10) NOT NULL REFERENCES hc.departments,
    facility_id    VARCHAR(10) NOT NULL REFERENCES hc.facilities,
    room_name      VARCHAR(80) NOT NULL
);

CREATE TABLE hc.beds (
    bed_id    VARCHAR(10) PRIMARY KEY,
    room_id   VARCHAR(10) NOT NULL REFERENCES hc.rooms,
    status    VARCHAR(20) NOT NULL
);

-- 1.3 Staff (doctors, nurses, radiographers and administrative staff in one tabl
CREATE TABLE hc.staff (
    staff_id          VARCHAR(10) PRIMARY KEY,
    first_name        VARCHAR(60),
    last_name         VARCHAR(60),
    role              VARCHAR(30) NOT NULL,
    department        VARCHAR(60),
    specialty         VARCHAR(60),
```

```
    facility_id       VARCHAR(10) NOT NULL REFERENCES hc.facilities,
    department_id     VARCHAR(10) NOT NULL REFERENCES hc.departments,
    years_experience  SMALLINT CHECK (years_experience BETWEEN 0 AND 60),
    status            VARCHAR(20)
);

-- 1.4 Optional patient master (NOT supplied; created empty so the patient querie
CREATE TABLE hc.patients (
    patient_id     VARCHAR(10) PRIMARY KEY,
    gender         VARCHAR(10),
    date_of_birth  DATE,
    blood_group    VARCHAR(5),
    phone_number   VARCHAR(20),
    province_id    VARCHAR(10) REFERENCES hc.provinces
);

-- 1.5 Clinical transactions ------------------------------------------------
CREATE TABLE hc.appointments (
    appointment_id    VARCHAR(12) PRIMARY KEY,
    patient_id        VARCHAR(10) NOT NULL,
    doctor_id         VARCHAR(10) NOT NULL REFERENCES hc.staff,
    facility_id       VARCHAR(10) NOT NULL REFERENCES hc.facilities,
    department_id     VARCHAR(10) NOT NULL REFERENCES hc.departments,
    appointment_date  DATE NOT NULL,
    visit_type        VARCHAR(20) NOT NULL CHECK (visit_type IN ('Consultation',
);

CREATE TABLE hc.diagnoses (
    diagnosis_id    VARCHAR(10) PRIMARY KEY,
    appointment_id  VARCHAR(12) NOT NULL REFERENCES hc.appointments,
    diagnosis       VARCHAR(60) NOT NULL,
    severity        VARCHAR(10) NOT NULL CHECK (severity IN ('Mild', 'Moderate',
);

CREATE TABLE hc.lab_tests (
    lab_id          VARCHAR(10) PRIMARY KEY,
    appointment_id  VARCHAR(12) NOT NULL REFERENCES hc.appointments,
    test            VARCHAR(40) NOT NULL,
    result          VARCHAR(10) NOT NULL CHECK (result IN ('Normal', 'Abnormal'))
);

CREATE TABLE hc.prescriptions (
    prescription_id  VARCHAR(10) PRIMARY KEY,
    appointment_id   VARCHAR(12) NOT NULL REFERENCES hc.appointments,
    medication       VARCHAR(60) NOT NULL,
    dosage           VARCHAR(40)
);
```

```
CREATE TABLE hc.admissions (
    admission_id          VARCHAR(10) PRIMARY KEY,
    appointment_id        VARCHAR(12) NOT NULL REFERENCES hc.appointments,
    length_of_stay_days   SMALLINT NOT NULL CHECK (length_of_stay_days > 0)
);

CREATE TABLE hc.referrals (
    referral_id           VARCHAR(10) PRIMARY KEY,
    appointment_id        VARCHAR(12) NOT NULL REFERENCES hc.appointments,
    referred_facility_id  VARCHAR(10) NOT NULL REFERENCES hc.facilities
);

-- 1.6 Billing, payments, insurance and feedback --------------------------------
CREATE TABLE hc.billing (
    billing_id      VARCHAR(12) PRIMARY KEY,
    appointment_id  VARCHAR(12) NOT NULL UNIQUE REFERENCES hc.appointments,
    amount          NUMERIC(14,2) NOT NULL CHECK (amount > 0),
    status          VARCHAR(20) NOT NULL CHECK (status IN ('Paid', 'Partially Pai
);

CREATE TABLE hc.payments (
    payment_id   VARCHAR(12) PRIMARY KEY,
    billing_id   VARCHAR(12) NOT NULL UNIQUE REFERENCES hc.billing,
    amount_paid  NUMERIC(14,2) NOT NULL CHECK (amount_paid >= 0),
    method       VARCHAR(20) NOT NULL,
    status       VARCHAR(20) NOT NULL
);

CREATE TABLE hc.insurance_claims (
    claim_id      VARCHAR(12) PRIMARY KEY,
    billing_id    VARCHAR(12) NOT NULL REFERENCES hc.billing,
    provider      VARCHAR(30) NOT NULL,
    claim_amount  NUMERIC(14,2) NOT NULL CHECK (claim_amount > 0),
    status        VARCHAR(20) NOT NULL
);

CREATE TABLE hc.patient_feedback (
    feedback_id     VARCHAR(12) PRIMARY KEY,
    appointment_id  VARCHAR(12) NOT NULL REFERENCES hc.appointments,
    rating          SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment         VARCHAR(40)
);

-- 1.7 Staging tables (raw copies of the files, no constraints) -----------------
CREATE TABLE stg.countries        (country_id TEXT, country_name TEXT, iso2 TEXT,
CREATE TABLE stg.provinces        (province_id TEXT, province_name TEXT, country_
```

```
CREATE TABLE stg.districts        (district_id TEXT, district_name TEXT, province
CREATE TABLE stg.sectors          (sector_id TEXT, sector_name TEXT, district_id
CREATE TABLE stg.villages         (village_id TEXT, village_name TEXT, sector_id
CREATE TABLE stg.departments      (department_id TEXT, department_name TEXT, faci
CREATE TABLE stg.specialties      (specialty_id TEXT, specialty TEXT, department_
CREATE TABLE stg.services         (service_id TEXT, service_name TEXT, department
CREATE TABLE stg.equipment        (equipment_id TEXT, equipment_name TEXT, depart
CREATE TABLE stg.rooms            (room_id TEXT, department_id TEXT, facility_id
CREATE TABLE stg.beds             (bed_id TEXT, room_id TEXT, status TEXT);
CREATE TABLE stg.staff            (staff_id TEXT, first_name TEXT, last_name TEXT
                                   specialty TEXT, facility_id TEXT, department_i
CREATE TABLE stg.appointments     (appointment_id TEXT, patient_id TEXT, doctor_i
                                   department_id TEXT, appointment_date DATE, vis
CREATE TABLE stg.diagnoses        (diagnosis_id TEXT, appointment_id TEXT, diagno
CREATE TABLE stg.lab_tests        (lab_id TEXT, appointment_id TEXT, test TEXT, r
CREATE TABLE stg.prescriptions    (prescription_id TEXT, appointment_id TEXT, med
CREATE TABLE stg.admissions       (admission_id TEXT, appointment_id TEXT, length
CREATE TABLE stg.referrals        (referral_id TEXT, appointment_id TEXT, referre
CREATE TABLE stg.billing          (billing_id TEXT, appointment_id TEXT, amount N
CREATE TABLE stg.payments         (payment_id TEXT, billing_id TEXT, amount_paid
CREATE TABLE stg.insurance_claims (claim_id TEXT, billing_id TEXT, provider TEXT,
CREATE TABLE stg.patient_feedback (feedback_id TEXT, appointment_id TEXT, rating


/* ============================================================================
   PART 2 | LOAD THE FILES INTO THE STAGING TABLES  (psql \copy; run inside csv_f
   ============================================================================

\copy stg.countries        FROM 'HCCountries.csv'              WITH (FORMAT csv,
\copy stg.provinces        FROM 'HC_Provinces.csv'             WITH (FORMAT csv,
\copy stg.districts        FROM 'HC_Districts.csv'             WITH (FORMAT csv,
\copy stg.sectors          FROM 'HC_Sectors.csv'               WITH (FORMAT csv,
\copy stg.villages         FROM 'HC_Villages.csv'              WITH (FORMAT csv,
\copy stg.departments      FROM 'HC_Departments.csv'           WITH (FORMAT csv,
\copy stg.specialties      FROM 'HC_Medical_Specialties.csv'   WITH (FORMAT csv,
\copy stg.services         FROM 'HC_Services.csv'              WITH (FORMAT csv,
\copy stg.equipment        FROM 'HC_Medical_Equipment.csv'     WITH (FORMAT csv,
\copy stg.rooms            FROM 'HC_Rooms.csv'                 WITH (FORMAT csv,
\copy stg.beds             FROM 'HC_Beds.csv'                  WITH (FORMAT csv,
\copy stg.staff            FROM 'HC_Doctor.csv'                WITH (FORMAT csv,
\copy stg.staff            FROM 'HC_Nurse.csv'                 WITH (FORMAT csv,
\copy stg.staff            FROM 'HC_Radiographer.csv'          WITH (FORMAT csv,
\copy stg.staff            FROM 'HC_Administrative_Staff.csv'  WITH (FORMAT csv,
\copy stg.appointments     FROM 'HC_Appointments.csv'          WITH (FORMAT csv,
\copy stg.diagnoses        FROM 'HC_Diagnoses_Clinical.csv'    WITH (FORMAT csv,
\copy stg.lab_tests        FROM 'HC_Lab_Tests.csv'             WITH (FORMAT csv,
\copy stg.prescriptions    FROM 'HC_Prescriptions.csv'         WITH (FORMAT csv,
```

```
\copy stg.admissions       FROM 'HC_Admissions.csv'           WITH (FORMAT csv,
\copy stg.referrals        FROM 'HC_Referrals.csv'            WITH (FORMAT csv,
\copy stg.billing          FROM 'HC_Billing.csv'              WITH (FORMAT csv,
\copy stg.payments         FROM 'HC_Payments.csv'             WITH (FORMAT csv,
\copy stg.insurance_claims FROM 'HC_Insurance_Claims.csv'     WITH (FORMAT csv,
\copy stg.patient_feedback FROM 'HC_Patient_Feedback.csv'     WITH (FORMAT csv,


/* ============================================================================
   PART 3 | BUILD THE CLEAN TABLES AND VALIDATE
   ============================================================================

-- 3.1 Populate hc tables from staging ------------------------------------------
-- Reference and capacity data load in full.
INSERT INTO hc.countries SELECT * FROM stg.countries;
INSERT INTO hc.provinces SELECT * FROM stg.provinces;
INSERT INTO hc.districts SELECT * FROM stg.districts;
INSERT INTO hc.sectors   SELECT * FROM stg.sectors;
INSERT INTO hc.villages  SELECT * FROM stg.villages;

INSERT INTO hc.facilities (facility_id)
SELECT DISTINCT facility_id FROM stg.departments ORDER BY facility_id;

INSERT INTO hc.departments SELECT department_id, department_name, facility_id FRO
INSERT INTO hc.specialties SELECT specialty_id, specialty, department_id     FROM
INSERT INTO hc.services    SELECT service_id, service_name, department_id     FRO
INSERT INTO hc.equipment   SELECT equipment_id, equipment_name, department_id, fa
INSERT INTO hc.rooms       SELECT room_id, department_id, facility_id, room_name
INSERT INTO hc.beds        SELECT bed_id, room_id, status FROM stg.beds;

INSERT INTO hc.staff
SELECT staff_id, first_name, last_name, role, department, specialty, facility_id,
       years_experience::SMALLINT, status
FROM stg.staff;

-- Appointments are the anchor table: all 5,000 rows load.
INSERT INTO hc.appointments
SELECT appointment_id, patient_id, doctor_id, facility_id, department_id, appoint
FROM stg.appointments;

-- Every other transaction table keeps only rows that link to one of those appoin
INSERT INTO hc.diagnoses
SELECT s.diagnosis_id, s.appointment_id, s.diagnosis, s.severity
FROM stg.diagnoses s JOIN hc.appointments a USING (appointment_id);

INSERT INTO hc.lab_tests
SELECT s.lab_id, s.appointment_id, s.test, s.result
```

```
FROM stg.lab_tests s JOIN hc.appointments a USING (appointment_id);

INSERT INTO hc.prescriptions
SELECT s.prescription_id, s.appointment_id, s.medication, s.dosage
FROM stg.prescriptions s JOIN hc.appointments a USING (appointment_id);

INSERT INTO hc.admissions
SELECT s.admission_id, s.appointment_id, s.length_of_stay_days
FROM stg.admissions s JOIN hc.appointments a USING (appointment_id);

INSERT INTO hc.referrals
SELECT s.referral_id, s.appointment_id, s.referred_facility_id
FROM stg.referrals s JOIN hc.appointments a USING (appointment_id);

INSERT INTO hc.billing
SELECT s.billing_id, s.appointment_id, s.amount, s.status
FROM stg.billing s JOIN hc.appointments a USING (appointment_id);

INSERT INTO hc.payments
SELECT s.payment_id, s.billing_id, s.amount_paid, s.method, s.status
FROM stg.payments s JOIN hc.billing b USING (billing_id);

INSERT INTO hc.insurance_claims
SELECT s.claim_id, s.billing_id, s.provider, s.claim_amount, s.status
FROM stg.insurance_claims s JOIN hc.billing b USING (billing_id);

INSERT INTO hc.patient_feedback
SELECT s.feedback_id, s.appointment_id, s.rating, s.comment
FROM stg.patient_feedback s JOIN hc.appointments a USING (appointment_id);

ANALYZE;

-- 3.2 Reconciliation: rows in the files versus rows linked to the 5,000 appointm
SELECT table_name,
       rows_in_file,
       rows_loaded,
       rows_in_file - rows_loaded                                    AS rows_not_l
       ROUND(100.0 * rows_loaded / rows_in_file, 1)                  AS pct_loaded
FROM (
    SELECT 'appointments'     AS table_name, (SELECT COUNT(*) FROM stg.appointmen
    SELECT 'diagnoses',                      (SELECT COUNT(*) FROM stg.diagnoses)
    SELECT 'lab_tests',                      (SELECT COUNT(*) FROM stg.lab_tests)
    SELECT 'prescriptions',                  (SELECT COUNT(*) FROM stg.prescripti
    SELECT 'admissions',                     (SELECT COUNT(*) FROM stg.admissions
    SELECT 'referrals',                      (SELECT COUNT(*) FROM stg.referrals)
    SELECT 'billing',                        (SELECT COUNT(*) FROM stg.billing),
    SELECT 'payments',                       (SELECT COUNT(*) FROM stg.payments),
```

```
    SELECT 'insurance_claims',              (SELECT COUNT(*) FROM stg.insurance_
    SELECT 'patient_feedback',              (SELECT COUNT(*) FROM stg.patient_fe
) t
ORDER BY table_name;

-- 3.3 Data-quality checks. Each row shows the number of records that FAIL the ru
SELECT check_no, check_description, records_failing,
       CASE WHEN records_failing = 0 THEN 'PASS' ELSE 'REVIEW' END AS result
FROM (
    SELECT 1 AS check_no, 'Duplicate appointment IDs in the source file' AS check
           (SELECT COUNT(*) - COUNT(DISTINCT appointment_id) FROM stg.appointment
    UNION ALL SELECT 2, 'Appointments with a missing patient, doctor, facility or
           (SELECT COUNT(*) FROM stg.appointments
             WHERE patient_id IS NULL OR doctor_id IS NULL OR facility_id IS NULL
    UNION ALL SELECT 3, 'Bills with an amount of zero or less (linked bills)',
           (SELECT COUNT(*) FROM hc.billing WHERE amount <= 0)
    UNION ALL SELECT 4, 'Ratings outside 1-5 (linked feedback)',
           (SELECT COUNT(*) FROM hc.patient_feedback WHERE rating NOT BETWEEN 1 A
    UNION ALL SELECT 5, 'Appointments without a bill',
           (SELECT COUNT(*) FROM hc.appointments a WHERE NOT EXISTS (SELECT 1 FRO
    UNION ALL SELECT 6, 'Bills without a payment record',
           (SELECT COUNT(*) FROM hc.billing b WHERE NOT EXISTS (SELECT 1 FROM hc.
    UNION ALL SELECT 7, 'Payments larger than the bill',
           (SELECT COUNT(*) FROM hc.payments p JOIN hc.billing b USING (billing_i
    UNION ALL SELECT 8, 'Claims larger than the bill',
           (SELECT COUNT(*) FROM hc.insurance_claims c JOIN hc.billing b USING (b
    UNION ALL SELECT 9, 'Payments whose status says "Paid" but where less than th
           (SELECT COUNT(*) FROM hc.payments p JOIN hc.billing b USING (billing_i
             WHERE p.status = 'Paid' AND p.amount_paid < b.amount)
    UNION ALL SELECT 10, 'Payments whose status says "Pending" but where money wa
           (SELECT COUNT(*) FROM hc.payments p WHERE p.status = 'Pending' AND p.a
    UNION ALL SELECT 11, 'Appointments whose department belongs to a different fa
           (SELECT COUNT(*) FROM hc.appointments a JOIN hc.departments d USING (d
             WHERE a.facility_id <> d.facility_id)
    UNION ALL SELECT 12, 'Payment status different from bill status',
           (SELECT COUNT(*) FROM hc.payments p JOIN hc.billing b USING (billing_i
) checks
ORDER BY check_no;
-- Reading the result: checks 9, 10 and 11 are expected to show REVIEW. They docu
-- limitations that are reported in the Week 7 testing report (see the notes in t


/* ============================================================================
   PART 4 | ANALYSIS QUERIES (the 40 business questions)
   ============================================================================

-- ================================== A. PATIENT ANALYSIS ====================
```

```sql
-- Q1. Retrieve all registered patients.
-- No patient master was supplied, so a patient is "registered" if they appear in
SELECT a.patient_id,
       MIN(a.appointment_date) AS first_visit,
       MAX(a.appointment_date) AS last_visit,
       COUNT(*)                AS visits
FROM hc.appointments a
GROUP BY a.patient_id
ORDER BY a.patient_id;

-- Q2. Count patients by gender.   [requires hc.patients]
SELECT COALESCE(gender, 'Unknown') AS gender,
       COUNT(*)                                          AS patients,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_of_patients
FROM hc.patients
GROUP BY COALESCE(gender, 'Unknown')
ORDER BY patients DESC;

-- Q3. Count patients by province.   [requires hc.patients with province_id]
SELECT COALESCE(pr.province_name, 'Unknown') AS province,
       COUNT(*)                              AS patients
FROM hc.patients p
LEFT JOIN hc.provinces pr USING (province_id)
GROUP BY COALESCE(pr.province_name, 'Unknown')
ORDER BY patients DESC;

-- Q4. Identify patients above 65 years.   [requires hc.patients with date_of_bir
SELECT patient_id,
       date_of_birth,
       DATE_PART('year', AGE(CURRENT_DATE, date_of_birth))::INT AS age_years
FROM hc.patients
WHERE DATE_PART('year', AGE(CURRENT_DATE, date_of_birth)) > 65
ORDER BY age_years DESC, patient_id;

-- Q5. Find patients with chronic diseases.
-- Chronic conditions in the data are Diabetes and Hypertension (analyst classifi
SELECT a.patient_id,
       STRING_AGG(DISTINCT d.diagnosis, ', ' ORDER BY d.diagnosis) AS chronic_con
       COUNT(*)                                                    AS chronic_vis
FROM hc.appointments a
JOIN hc.diagnoses d USING (appointment_id)
WHERE d.diagnosis IN ('Diabetes', 'Hypertension')
GROUP BY a.patient_id
ORDER BY chronic_visits DESC, a.patient_id;

-- Q6. Retrieve patients covered by insurance.
```

```sql
-- Coverage is taken as "has at least one insurance claim on a bill".
SELECT a.patient_id,
       COUNT(DISTINCT c.claim_id)       AS claims,
       SUM(c.claim_amount)              AS total_claimed_rwf,
       STRING_AGG(DISTINCT c.provider, ', ' ORDER BY c.provider) AS providers
FROM hc.insurance_claims c
JOIN hc.billing b      USING (billing_id)
JOIN hc.appointments a USING (appointment_id)
GROUP BY a.patient_id
ORDER BY total_claimed_rwf DESC;

-- Q7. Count patients by blood group.   [requires hc.patients]
SELECT COALESCE(blood_group, 'Unknown') AS blood_group,
       COUNT(*)                         AS patients
FROM hc.patients
GROUP BY COALESCE(blood_group, 'Unknown')
ORDER BY patients DESC;

-- Q8. List the youngest and oldest patients.   [requires hc.patients with date_o
(SELECT 'Youngest' AS category, patient_id, date_of_birth,
        DATE_PART('year', AGE(CURRENT_DATE, date_of_birth))::INT AS age_years
 FROM hc.patients WHERE date_of_birth IS NOT NULL
 ORDER BY date_of_birth DESC LIMIT 5)
UNION ALL
(SELECT 'Oldest', patient_id, date_of_birth,
        DATE_PART('year', AGE(CURRENT_DATE, date_of_birth))::INT
 FROM hc.patients WHERE date_of_birth IS NOT NULL
 ORDER BY date_of_birth ASC LIMIT 5);

-- Q9. Identify duplicate phone numbers.   [requires hc.patients]
SELECT phone_number,
       COUNT(*)                                      AS patients_sharing_number,
       STRING_AGG(patient_id, ', ' ORDER BY patient_id) AS patient_ids
FROM hc.patients
WHERE phone_number IS NOT NULL
GROUP BY phone_number
HAVING COUNT(*) > 1
ORDER BY patients_sharing_number DESC, phone_number;

-- Q10. Calculate average patient age.   [requires hc.patients with date_of_birth
SELECT COUNT(*)
       ROUND(AVG(DATE_PART('year', AGE(CURRENT_DATE, date_of_birth)))::NUMERIC, 1
FROM hc.patients
WHERE date_of_birth IS NOT NULL;


-- ================================== B. APPOINTMENT ANALYSIS ================
```

```sql
-- Q11. Count appointments by month, with month-on-month growth.
WITH monthly AS (
    SELECT DATE_TRUNC('month', appointment_date)::DATE AS month,
           COUNT(*)                                    AS appointments
    FROM hc.appointments
    GROUP BY 1
)
SELECT month,
       appointments,
       ROUND(100.0 * (appointments - LAG(appointments) OVER (ORDER BY month))
             / NULLIF(LAG(appointments) OVER (ORDER BY month), 0), 1) AS growth_p
FROM monthly
ORDER BY month;

-- Q12. Identify the busiest hospitals (by number of appointments).
SELECT facility_id,
       COUNT(*)                                          AS appointments,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_of_all,
       RANK() OVER (ORDER BY COUNT(*) DESC)              AS busiest_rank
FROM hc.appointments
GROUP BY facility_id
ORDER BY busiest_rank, facility_id;

-- Q13. Find the busiest departments (departments with the same name are combined
SELECT d.department_name,
       COUNT(*)                                          AS appointments,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_of_all,
       RANK() OVER (ORDER BY COUNT(*) DESC)              AS busiest_rank
FROM hc.appointments a
JOIN hc.departments d USING (department_id)
GROUP BY d.department_name
ORDER BY busiest_rank, d.department_name;

-- Q14. Find the doctors with the highest workload (top 10).
SELECT s.staff_id                     AS doctor_id,
       s.first_name || ' ' || s.last_name AS doctor_name,
       s.department,
       s.facility_id,
       COUNT(*)                       AS appointments
FROM hc.appointments a
JOIN hc.staff s ON s.staff_id = a.doctor_id
GROUP BY s.staff_id, s.first_name, s.last_name, s.department, s.facility_id
ORDER BY appointments DESC, s.staff_id
LIMIT 10;

-- Q15. Identify referral patterns.
```

```sql
-- 15a. Busiest referral routes (origin facility to receiving facility).
SELECT a.facility_id              AS referred_from,
       r.referred_facility_id     AS referred_to,
       COUNT(*)                   AS referrals
FROM hc.referrals r
JOIN hc.appointments a USING (appointment_id)
GROUP BY a.facility_id, r.referred_facility_id
ORDER BY referrals DESC, referred_from, referred_to
LIMIT 15;

-- 15b. Referral rate by originating facility.
SELECT a.facility_id,
       COUNT(*)                                          AS appointments,
       COUNT(r.referral_id)                              AS referrals,
       ROUND(100.0 * COUNT(r.referral_id) / COUNT(*), 1)   AS referral_rate_pct
FROM hc.appointments a
LEFT JOIN hc.referrals r USING (appointment_id)
GROUP BY a.facility_id
ORDER BY referral_rate_pct DESC, a.facility_id;

-- Q16. Count emergency appointments.
SELECT COUNT(*) FILTER (WHERE visit_type = 'Emergency')                        A
       COUNT(*)                                                                A
       ROUND(100.0 * COUNT(*) FILTER (WHERE visit_type = 'Emergency') / COUNT(*),
FROM hc.appointments;

-- Q17. Calculate average appointments per doctor.
WITH per_doctor AS (
    SELECT s.staff_id, COUNT(a.appointment_id) AS appointments
    FROM hc.staff s
    LEFT JOIN hc.appointments a ON a.doctor_id = s.staff_id
    WHERE s.role = 'Doctor'
    GROUP BY s.staff_id
)
SELECT COUNT(*)                                AS doctors,
       SUM(appointments)                       AS appointments,
       ROUND(AVG(appointments), 2)             AS avg_appointments_per_doctor,
       MIN(appointments)                       AS fewest,
       MAX(appointments)                       AS most
FROM per_doctor;

-- Q18. Retrieve appointments by province.
-- The supplied data has no link from facilities or patients to a province, so th
-- available breakdown (appointments by facility) is returned.
SELECT facility_id, COUNT(*) AS appointments
FROM hc.appointments
GROUP BY facility_id
```

```
ORDER BY appointments DESC, facility_id;

-- Q19. Identify facilities with the highest patient volume (distinct patients).
SELECT facility_id,
       COUNT(DISTINCT patient_id)                      AS distinct_patients,
       COUNT(*)                                        AS appointments,
       RANK() OVER (ORDER BY COUNT(DISTINCT patient_id) DESC) AS volume_rank
FROM hc.appointments
GROUP BY facility_id
ORDER BY volume_rank, facility_id;

-- Q20. Find peak consultation periods.
-- 20a. By month.
SELECT TO_CHAR(appointment_date, 'YYYY-MM') AS month,
       COUNT(*)                             AS consultations
FROM hc.appointments
WHERE visit_type = 'Consultation'
GROUP BY 1
ORDER BY consultations DESC, month;

-- 20b. By day of the week.
SELECT TO_CHAR(appointment_date, 'FMDay')   AS weekday,
       COUNT(*)                             AS consultations
FROM hc.appointments
WHERE visit_type = 'Consultation'
GROUP BY TO_CHAR(appointment_date, 'FMDay'), EXTRACT(ISODOW FROM appointment_date
ORDER BY consultations DESC;


-- ===================================== C. CLINICAL ANALYSIS ==================

-- Q21. Top diagnoses (up to 20; the data contains five).
SELECT diagnosis,
       COUNT(*)                                          AS cases,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_of_cases
FROM hc.diagnoses
GROUP BY diagnosis
ORDER BY cases DESC, diagnosis
LIMIT 20;

-- Q22. Most requested laboratory tests.
SELECT test,
       COUNT(*)                                          AS tests_requested,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_of_tests
FROM hc.lab_tests
GROUP BY test
ORDER BY tests_requested DESC, test;
```

```
-- Q23. Most prescribed medications.
SELECT medication,
       COUNT(*)                                          AS prescriptions,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_of_prescriptions
FROM hc.prescriptions
GROUP BY medication
ORDER BY prescriptions DESC, medication;

-- Q24. Average admission duration (overall and by diagnosis).
SELECT COALESCE(d.diagnosis, 'ALL DIAGNOSES')   AS diagnosis,
       COUNT(*)                                 AS admissions,
       ROUND(AVG(ad.length_of_stay_days), 2)    AS avg_length_of_stay_days
FROM hc.admissions ad
JOIN hc.diagnoses d USING (appointment_id)
GROUP BY ROLLUP (d.diagnosis)
ORDER BY (d.diagnosis IS NULL), d.diagnosis;

-- Q25. Disease severity distribution (share of each severity within a diagnosis)
SELECT diagnosis,
       severity,
       COUNT(*)                                                                AS
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (PARTITION BY diagnosis), 1) A
FROM hc.diagnoses
GROUP BY diagnosis, severity
ORDER BY diagnosis, CASE severity WHEN 'Mild' THEN 1 WHEN 'Moderate' THEN 2 ELSE

-- Q26. Laboratory test utilisation (tests per test type, abnormal rate, and shar
SELECT test,
       COUNT(*)                                                          AS test
       COUNT(*) FILTER (WHERE result = 'Abnormal')                       AS abno
       ROUND(100.0 * COUNT(*) FILTER (WHERE result = 'Abnormal') / COUNT(*), 1) A
FROM hc.lab_tests
GROUP BY test
ORDER BY tests DESC;

SELECT COUNT(DISTINCT l.appointment_id)
       (SELECT COUNT(*) FROM hc.appointments)
       ROUND(100.0 * COUNT(DISTINCT l.appointment_id) / (SELECT COUNT(*) FROM hc.
FROM hc.lab_tests l;

-- Q27. Prescription trends (monthly volume by medication).
SELECT TO_CHAR(a.appointment_date, 'YYYY-MM') AS month,
       p.medication,
       COUNT(*)                                AS prescriptions
FROM hc.prescriptions p
JOIN hc.appointments a USING (appointment_id)
```

```
GROUP BY 1, p.medication
ORDER BY month, prescriptions DESC, p.medication;

-- Q28. Referral destinations.
SELECT referred_facility_id                        AS destination_facility,
       COUNT(*)                                    AS referrals,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_of_referrals
FROM hc.referrals
GROUP BY referred_facility_id
ORDER BY referrals DESC, destination_facility;

-- Q29. Admission rate by diagnosis.
SELECT d.diagnosis,
       COUNT(*)                                            AS appointments,
       COUNT(ad.admission_id)                              AS admissions,
       ROUND(100.0 * COUNT(ad.admission_id) / COUNT(*), 2) AS admission_rate_pct
FROM hc.diagnoses d
LEFT JOIN hc.admissions ad USING (appointment_id)
GROUP BY d.diagnosis
ORDER BY admission_rate_pct DESC;

-- Q30. Clinical workload by department.
WITH appts AS (
    SELECT a.department_id, COUNT(*) AS appointments
    FROM hc.appointments a GROUP BY a.department_id
), labs AS (
    SELECT a.department_id, COUNT(*) AS lab_tests
    FROM hc.lab_tests l JOIN hc.appointments a USING (appointment_id) GROUP BY a.
), rx AS (
    SELECT a.department_id, COUNT(*) AS prescriptions
    FROM hc.prescriptions p JOIN hc.appointments a USING (appointment_id) GROUP B
), adm AS (
    SELECT a.department_id, COUNT(*) AS admissions
    FROM hc.admissions ad JOIN hc.appointments a USING (appointment_id) GROUP BY
)
SELECT d.department_name,
       SUM(COALESCE(appts.appointments, 0))   AS appointments,
       SUM(COALESCE(labs.lab_tests, 0))       AS lab_tests,
       SUM(COALESCE(rx.prescriptions, 0))     AS prescriptions,
       SUM(COALESCE(adm.admissions, 0))       AS admissions
FROM hc.departments d
LEFT JOIN appts USING (department_id)
LEFT JOIN labs  USING (department_id)
LEFT JOIN rx    USING (department_id)
LEFT JOIN adm   USING (department_id)
GROUP BY d.department_name
ORDER BY appointments DESC, d.department_name;
```

```
-- ================================== D. FINANCIAL ANALYSIS ================
-- Billed amount is the revenue measure; cash received comes from payments.amount

-- Q31. Revenue by facility.
SELECT a.facility_id,
       COUNT(*)                                                    AS bills,
       SUM(b.amount)                                               AS billed_rwf,
       SUM(p.amount_paid)                                          AS received_rwf,
       SUM(b.amount) - SUM(p.amount_paid)                          AS outstanding_r
       ROUND(100.0 * SUM(p.amount_paid) / SUM(b.amount), 1)        AS collection_ra
FROM hc.billing b
JOIN hc.appointments a USING (appointment_id)
JOIN hc.payments p     USING (billing_id)
GROUP BY a.facility_id
ORDER BY billed_rwf DESC;

-- Q32. Revenue by department (departments with the same name are combined).
SELECT d.department_name,
       SUM(b.amount)                                          AS billed_rwf,
       SUM(p.amount_paid)                                     AS received_rwf,
       ROUND(100.0 * SUM(b.amount) / SUM(SUM(b.amount)) OVER (), 1) AS pct_of_rev
FROM hc.billing b
JOIN hc.appointments a USING (appointment_id)
JOIN hc.departments d  ON d.department_id = a.department_id
JOIN hc.payments p     USING (billing_id)
GROUP BY d.department_name
ORDER BY billed_rwf DESC;

-- Q33. Revenue by service.
-- Bills are not linked to individual services in the supplied data, so revenue i
-- service line (visit type), the closest available classification.
SELECT a.visit_type                                       AS service_line,
       COUNT(*)                                           AS bills,
       SUM(b.amount)                                      AS billed_rwf,
       ROUND(AVG(b.amount), 0)                            AS avg_bill_rwf,
       ROUND(100.0 * SUM(b.amount) / SUM(SUM(b.amount)) OVER (), 1) AS pct_of_rev
FROM hc.billing b
JOIN hc.appointments a USING (appointment_id)
GROUP BY a.visit_type
ORDER BY billed_rwf DESC;

-- Q34. Insurance claims by provider.
SELECT provider,
       COUNT(*)                                                        AS claim
       SUM(claim_amount)                                               AS claim
```

```
       COUNT(*) FILTER (WHERE status = 'Approved')                    AS appro
       COUNT(*) FILTER (WHERE status = 'Pending')                     AS pendi
       ROUND(100.0 * COUNT(*) FILTER (WHERE status = 'Approved') / COUNT(*), 1) A
FROM hc.insurance_claims
GROUP BY provider
ORDER BY claimed_rwf DESC;

-- Q35. Outstanding payments.
-- 35a. Cash basis: billed amount minus amount paid.
SELECT SUM(b.amount)                                          AS billed_rwf,
       SUM(p.amount_paid)                                     AS received_rwf,
       SUM(b.amount) - SUM(p.amount_paid)                     AS outstanding_rwf,
       ROUND(100.0 * (SUM(b.amount) - SUM(p.amount_paid)) / SUM(b.amount), 1) AS
FROM hc.billing b
JOIN hc.payments p USING (billing_id);

-- 35b. Status basis (for comparison): bills not marked "Paid". This differs shar
--      basis because payment status does not reflect the amounts actually paid (
SELECT b.status                                               AS bill_status,
       COUNT(*)                                               AS bills,
       SUM(b.amount)                                          AS billed_rwf,
       SUM(p.amount_paid)                                     AS received_rwf,
       ROUND(100.0 * SUM(p.amount_paid) / SUM(b.amount), 1) AS received_pct_of_bi
FROM hc.billing b
JOIN hc.payments p USING (billing_id)
GROUP BY b.status
ORDER BY billed_rwf DESC;

-- Q36. Average billing amount.
-- 36a. Overall.
SELECT COUNT(*)                                                    AS bills,
       ROUND(AVG(amount), 0)                                       AS avg_bill_rwf,
       ROUND((PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount))::NUMERIC, 0) A
       MIN(amount)                                                 AS smallest_rwf,
       MAX(amount)                                                 AS largest_rwf
FROM hc.billing;

-- 36b. By visit type.
SELECT a.visit_type,
       COUNT(*)                  AS bills,
       ROUND(AVG(b.amount), 0)   AS avg_bill_rwf
FROM hc.billing b
JOIN hc.appointments a USING (appointment_id)
GROUP BY a.visit_type
ORDER BY avg_bill_rwf DESC;

-- Q37. Monthly revenue trend (with month-on-month growth and running total).
```

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', a.appointment_date)::DATE AS month,
           SUM(b.amount)                                 AS billed_rwf,
           SUM(p.amount_paid)                            AS received_rwf
    FROM hc.billing b
    JOIN hc.appointments a USING (appointment_id)
    JOIN hc.payments p     USING (billing_id)
    GROUP BY 1
)
SELECT month,
       billed_rwf,
       received_rwf,
       ROUND(100.0 * (billed_rwf - LAG(billed_rwf) OVER (ORDER BY month))
             / NULLIF(LAG(billed_rwf) OVER (ORDER BY month), 0), 1) AS billed_gro
       SUM(billed_rwf) OVER (ORDER BY month)                       AS billed_run
FROM monthly
ORDER BY month;

-- Q38. Payment methods.
SELECT method,
       COUNT(*)                                          AS payments,
       SUM(amount_paid)                                  AS received_rwf,
       ROUND(100.0 * SUM(amount_paid) / SUM(SUM(amount_paid)) OVER (), 1) AS pct_
FROM hc.payments
GROUP BY method
ORDER BY received_rwf DESC;

-- Q39. Top revenue-generating hospitals (top 10 by billed amount).
SELECT a.facility_id,
       SUM(b.amount)                                     AS billed_rwf,
       ROUND(100.0 * SUM(b.amount) / SUM(SUM(b.amount)) OVER (), 1) AS pct_of_rev
FROM hc.billing b
JOIN hc.appointments a USING (appointment_id)
GROUP BY a.facility_id
ORDER BY billed_rwf DESC
LIMIT 10;

-- Q40. Financial performance ranking (billed amount and collection rate, equally
WITH facility_finance AS (
    SELECT a.facility_id,
           SUM(b.amount)                          AS billed_rwf,
           SUM(p.amount_paid)                     AS received_rwf,
           SUM(p.amount_paid) / SUM(b.amount)     AS collection_rate
    FROM hc.billing b
    JOIN hc.appointments a USING (appointment_id)
    JOIN hc.payments p     USING (billing_id)
    GROUP BY a.facility_id
```

```
), ranked AS (
    SELECT facility_id, billed_rwf, received_rwf, collection_rate,
           RANK() OVER (ORDER BY billed_rwf DESC)      AS rank_by_billed,
           RANK() OVER (ORDER BY collection_rate DESC) AS rank_by_collection
    FROM facility_finance
)
SELECT facility_id,
       billed_rwf,
       ROUND(100 * collection_rate, 1)                  AS collection_rate_pct,
       rank_by_billed,
       rank_by_collection,
       RANK() OVER (ORDER BY (rank_by_billed + rank_by_collection), rank_by_bille
FROM ranked
ORDER BY overall_rank, facility_id;


/* ============================================================================
   PART 5 | KPI RECONCILIATION WITH THE POWER BI DASHBOARD
   Each value below should match the dashboard card or total it is named after.
   ============================================================================

SELECT kpi, value
FROM (
    SELECT  1 AS sort_order, 'Total appointments'          AS kpi, (SELECT COUNT(
    UNION ALL SELECT  2, 'Unique patients',                (SELECT COUNT(DISTINCT
    UNION ALL SELECT  3, 'Total billed (RWF)',             (SELECT SUM(amount) FR
    UNION ALL SELECT  4, 'Average bill (RWF)',             (SELECT ROUND(AVG(amou
    UNION ALL SELECT  5, 'Cash received (RWF)',            (SELECT SUM(amount_pai
    UNION ALL SELECT  6, 'Cash outstanding (RWF)',         (SELECT SUM(b.amount)
    UNION ALL SELECT  7, 'Collection rate (%)',            (SELECT ROUND(100.0 *
    UNION ALL SELECT  8, 'Insurance claims',               (SELECT COUNT(*) FROM
    UNION ALL SELECT  9, 'Claim approval rate (%)',        (SELECT ROUND(100.0 *
    UNION ALL SELECT 10, 'Admissions',                     (SELECT COUNT(*) FROM
    UNION ALL SELECT 11, 'Admission rate (%)',             (SELECT ROUND(100.0 *
    UNION ALL SELECT 12, 'Average length of stay (days)',  (SELECT ROUND(AVG(leng
    UNION ALL SELECT 13, 'Lab tests',                      (SELECT COUNT(*) FROM
    UNION ALL SELECT 14, 'Abnormal lab results (%)',       (SELECT ROUND(100.0 *
    UNION ALL SELECT 15, 'Prescriptions',                  (SELECT COUNT(*) FROM
    UNION ALL SELECT 16, 'Referrals',                      (SELECT COUNT(*) FROM
    UNION ALL SELECT 17, 'Doctors',                        (SELECT COUNT(*) FROM
    UNION ALL SELECT 18, 'Total staff',                    (SELECT COUNT(*) FROM
    UNION ALL SELECT 19, 'Appointments per doctor',        (SELECT ROUND((SELECT
    UNION ALL SELECT 20, 'Average satisfaction rating',    (SELECT ROUND(AVG(rati
    UNION ALL SELECT 21, 'Feedback responses',             (SELECT COUNT(*) FROM
) k
ORDER BY sort_order;
```

```
-- End of script
```